# Error 404
# [group 29]

## Project Phase 2 – Database Design

(CMPG311)

**Group Members:**

[44565720 – M. Mabunda] – Group Leader

[44222564 – A. Makhubele]

[49958623 – C. Sujee]

[50609173 – P. Vermeulen]

[42103789 – M. Xoko]

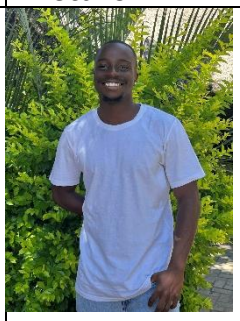
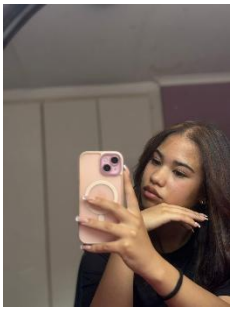
Submission date: 17/04/2026

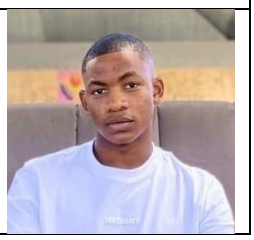
# Contents

# Project Phase 1 – Database Initial Study

## 1. MEMBERS OF THE GROUP

## Member Information

| Name & Surname | Student Number | Cell Phone Number | Picture |
|---|---|---|---|
| Mbhoni Mabunda (Group Leader) | 44565720 | 060 483 5497 |  |
| Chavonne Sujee | 49958623 | 065 998 7788 |  |
| PJ Vermeulen | 50609173 | 072 090 4589 |  |

| | | | |
|---|---|---|---|
| Andziso Makhubele | 44222564 | 071 487 1705 |  |
| Mihlayonke Xoko | 42103789 | 073 054 3641 |  |

# 2. ANALYSE COMPANY SITUATION

## 1. Objectives of the Company

The main goal of our gym is to offer good fitness services to its members while keeping things organized behind the scenes. To do that, our gym needs a system that can manage memberships, personal trainers, classes, and payments in one place instead of relying on scattered records.

These are some of our key goals:

- Keep member information accurate and easy to access
- Track membership plans and how long each membership lasts
- Store personal trainer details and manage their schedules
- Organize fitness classes and keep track of bookings
- Record payments and keep a history of proof of payment
- Reduce the amount of manual paperwork the staff deals with
- Give management access to reports like membership numbers, personal trainer's schedules, and payment records
- Track gym attendance

## 2. Operations of the Company

The gym runs several day-to-day activities, and most of them involve handling a lot of information. Having a proper system helps keep everything running smoothly.

### Member Registration

When someone joins the gym, they need to register first. They provide basic details like their name, contact information, and the membership plan they want. All of this information gets stored in the system so the staff can access it when needed.

### Membership Management

Members can choose different membership options, such as monthly, quarterly, or yearly plans. The system keeps track of when a membership starts, when it expires, and whether it needs to be renewed.

### Personal Trainer Management

The gym employs personal trainers who help members with workouts and run classes. Their information, areas of expertise, and work schedules are stored in the system so everything stays organized.

### Class Scheduling

The gym offers different types of classes. Yoga, spinning, cardio sessions, strength training, and so on. Members can book a spot in these classes depending on the personal trainer's availability and how many people the class can handle.

### Payment Processing

Members need to pay for memberships and sometimes for personal training sessions or classes. The system records each payment, including the amount, the date, and the method used.

### Equipment and Facility Usage

Gym equipment and facilities also need to be monitored so they stay available and in good condition. While this project won't manage equipment in detail, it's still something the gym keeps an eye on.

### Gym Attendance

The system records and analyses member attendance at the gym. It identifies peak hours when the gym reaches full capacity so that members who prefer quieter time slots can choose alternative times to visit. The system also tracks the most popular classes and maintains a gym attendance streak for each member, indicating how consistently they attend the gym. Based on attendance consistency, members are awarded star rankings.

# 3. DEFINE PROBLEMS AND CONSTRAINTS

## 1. Problem

a) Overcrowding - leads to relying on paper trails and spreadsheets which produces errors and slow processing, and this requires several hours to generate reports.
b) Poor communication - shift changes between staff can lead to misunderstandings regarding class schedules, payments, etc
c) Lack of integrated system - causes conflict in training schedules
d) Data security - unauthorized staff can access member data and personal information.
e) Maintaining memberships - members cancel before contract has ended.
f) Gym understaffed – overtime will be needed and staff are paid according to schedule / not enough personal trainers provided according to member demand.
g) Data redundancy - same member but different contact information.

## 2. Constraints

a) Budget constraint – project must stay within budget of R300 000, covering gym equipment, hardware and software
b) Technical constraint – update hardware and software, database must use SQL developer
c) Operational constraint – staff members should be able to use system through training
d) Security constraint – the database must be encrypted, access control, data backup and restoration if needed
e) Scalability constraint – lack of real time data, system must be designed for future growth
f) Time constraint – limited time to implement new system and train existing and potential staff

## 3. Structure of the Company

The gym has a simple structure where different people handle different responsibilities.

### Gym Manager

The manager is responsible for overseeing the entire operation. This includes supervising staff, checking membership activity, and reviewing the overall performance of the business.

### Personal Trainers

Personal trainers work directly with members. They guide workouts, help with fitness goals, and lead various classes. Each personal trainer usually has a specific area they specialize in.

### Reception Staff

Reception staff handle most of the front desk work. They register new members, process renewals, book classes, and deal with payments.

### Members

Members are the customers of the gym. They sign up for memberships, attend classes, and use the gym's facilities.

The database system will support all of these roles by making information easy to find and keeping records up to date.

# 4. DATABASE SYSTEM SPECIFICATION

Right now, our gym is dealing with a few operational problems. Things like overcrowding, poor communication between staff, duplicate data, and the lack of one central system to manage everything. A lot of the information is still handled through paper records or spreadsheets. Because of that, it takes time to find information or generate reports, and mistakes can easily happen. For example, the same member might be recorded more than once with slightly different details.

To fix these issues, our gym plans to implement a centralized database system using Oracle SQL Developer. The idea is to keep all the important information in one place so staff can access accurate and updated data whenever they need it. This should cut down on errors, improve communication between staff members, and generally make daily tasks easier to manage.

The database will mainly focus on managing members, memberships, personal trainers, classes, payments, and gym attendance. Reception staff will use the system to register new members, update their details, and record payments. Staff will also be able to manage class schedules and assign personal trainers without running into scheduling conflicts.

Tracking attendance will also be part of the system. Each time members visit our gym, their attendance will be recorded. This helps management see the peak hours of our gym and which classes are the most popular. With that kind of information, our gym can adjust schedules and hopefully avoid overcrowding during peak hours.

Since the system will store personal information, security is also a priority. Access to the database will be controlled so that only authorized staff members can view or update certain records. Regular backups will also be done to make sure data is not lost if something goes wrong.

## Information Requirements

For the system to work properly, it needs to store several types of information related to the gym's daily operations.

First, the database will store member information. This includes the member ID, name, contact details, address, membership type, and the start and end dates of the membership. This makes it easier for staff to track active members and manage renewals.

The system will also store personal trainer details such as trainer ID, name, contact information, specialization, and work schedules. Keeping this information organized helps avoid conflicts when personal trainers are assigned to classes or training sessions.

Details about membership plans also need to be recorded. This includes the membership ID, the type of plan, price, duration, and any benefits that come with it. That way, members can choose the option that suits them best.

The database will also manage fitness classes. Information such as class ID, class name, assigned personal trainer, schedule, and the maximum number of people allowed will be stored. Members will be able to book classes, and the system will keep track of those bookings.

Along with that, the system will store class booking records, which include the booking ID, member ID, class ID, and booking date.

Another key part is payment information. The database will record details like payment ID, member ID, payment amount, payment date, and the payment method used. This helps staff confirm that memberships are paid and still active.

And lastly, the system will record gym attendance. It will track when members enter the gym or attend classes. Over time, this data can show attendance patterns and member activity levels.

## Scope of the Project

This project focuses on designing and building a database that supports our gym's daily operations. The main goal is to manage information related to members, personal trainers, memberships, classes, bookings, payments, and attendance.

The system will allow staff to register new members, update memberships, schedule classes, assign personal trainers, record payments, and monitor attendance. It will also help management generate reports and review things like membership numbers, personal trainer schedules, and popular classes.

The database will be created using Oracle SQL Developer, as required for the course project.

The following are functional requirements:

- Maintain Member records
- Maintain Membership Plans
- Maintain Personal Trainer records
- Maintain Class information
- Maintain Bookings
- Maintain Payment records
- Record gym attendance
- Schedule classes and assign trainers
- Monitor membership status and gym attendance
- Provide member and class information
- Generate membership, payment, and attendance reports

## Boundaries of the Project

Even though the system will help manage many parts of the gym's operations, there are still some limits to what will be included.

The project mainly focuses on storing and managing data, so it won't include a full mobile or web application. Equipment maintenance will also not be handled in detail within this system.

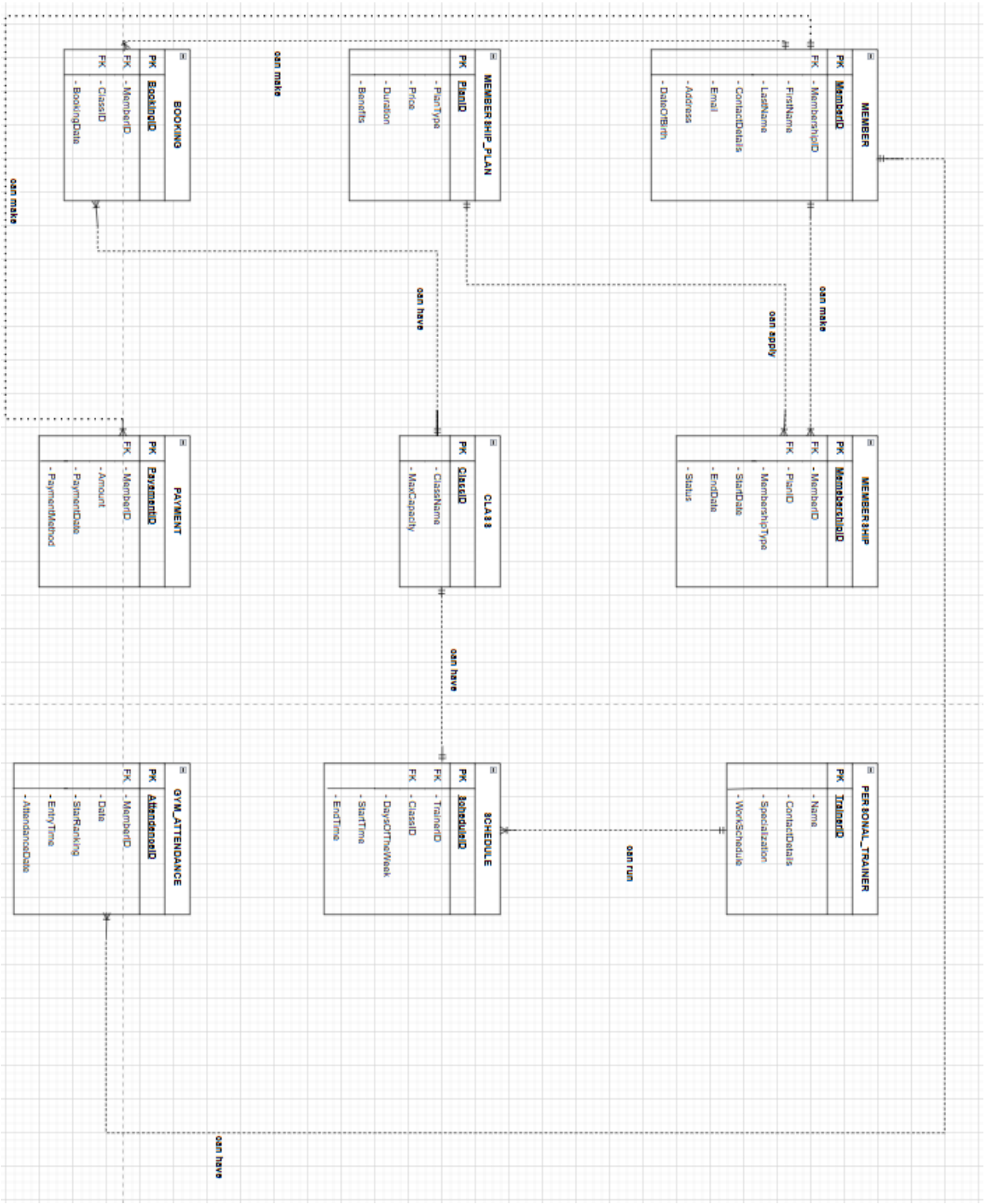
Some advanced features are outside the scope of our project as well. For example, things like biometric entry systems or online payment gateway integration won't be included. The database will also represent only one gym branch, instead of multiple locations.

# Project Phase 2 – Database Design

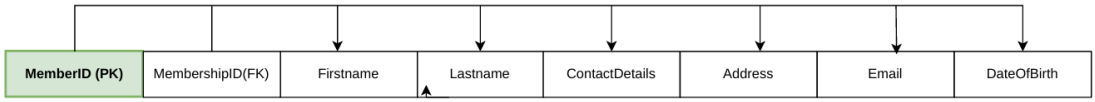
## 1. BUSINESS RULES

- One member can have many memberships over time
- Many memberships can be made by one member over time
- One member can make many payments
- Many payments are made by one member
- One membership plan can apply to many memberships
- Many memberships belong to one membership plan
- One personal trainer can run many schedules
- Many schedules are run by one personal trainer
- One class can have one schedule
- One schedule can have one class
- One class can have many bookings
- Many bookings belong to one class
- One member can make many bookings
- Many bookings are made by one member
- One member can have many attendance records
- Many attendance records belong to one member

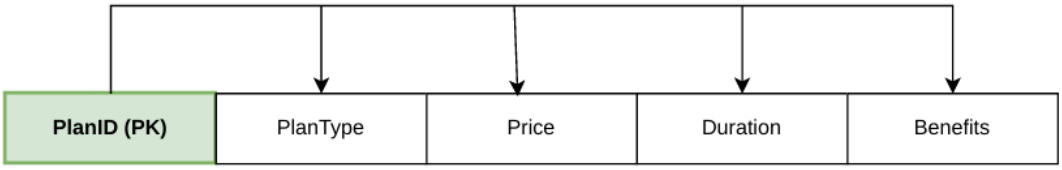
# 2. ER DIAGRAM


MEMBER
PK MemberID
FK - MembershipID
- FirstName
- LastName
- ContactDetails
- Email
- Address
- DateOfBirth
MEMBERSHIP
PK MembershipID
FK - MemberID
FK - PlanID
- MembershipType
- StartDate
- EndDate
- Status
MEMBERSHIP_PLAN
PK PlanID
- PlanType
- Price
- Duration
- Benefits
BOOKING
PK BookingID
FK - MemberID
FK - ClassID
- BookingDate
PAYMENT
PK PaymentID
FK - MemberID
- Amount
- PaymentDate
- PaymentMethod
CLASS
PK ClassID
- ClassName
- MaxCapacity
SCHEDULE
PK ScheduleID
FK - TrainerID
FK - ClassID
- DaysOfTheWeek
- StartTime
- EndTime
PERSONAL_TRAINER
PK TrainerID
- Name
- ContactDetails
- Specialization
- WorkSchedule
GYM_ATTENDANCE
PK AttendanceID
FK - MemberID
- Date
- StarRanking
- EntryTime
- AttendanceDate
can make
can make
can apply
can have
can have
can run
can have

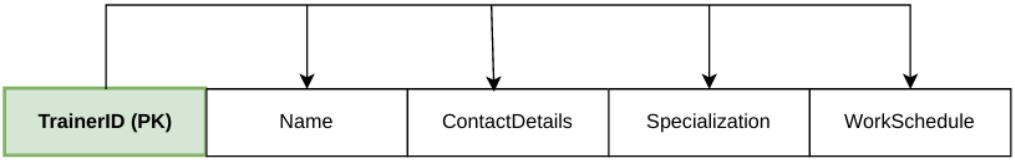
# 3. DEPENDENCY DIAGRAMS



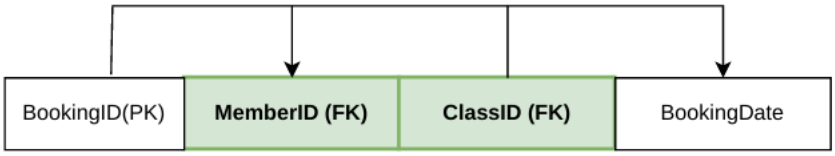

MEMBER(MEMBER_ID,MEMBERSHIP_ID,FIRSTNAME,LASTNAME,CONTACT_DETAILS,ADRESS,EMAIL.DOB)



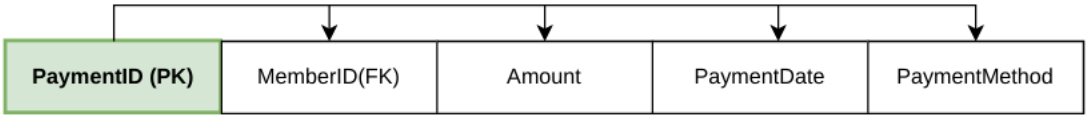

MEMBERSHIP_PLAN(PLAN_ID,PLAN_TYPE,PRICE,DURATION,BENFITS)



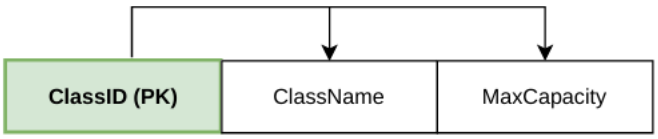

PERSONAL_TRAINER(TRAINER_ID,NAME,CONTACT_DETAILS,SPECIALIZATION,WORK_SCHEDULE)



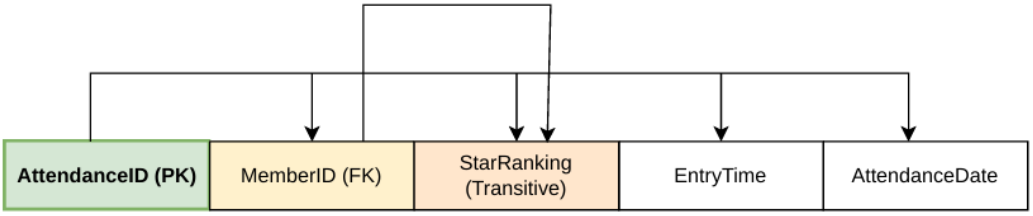

BOOKING(BOOKING_ID,MEMBER_ID,CLASS_ID,BOOKING_DATE)




PAYMENT(PAYMENT_ID,MEMBER_ID,AMOUNT,PAYMENT_DATE,PAYMENT_METHOD)

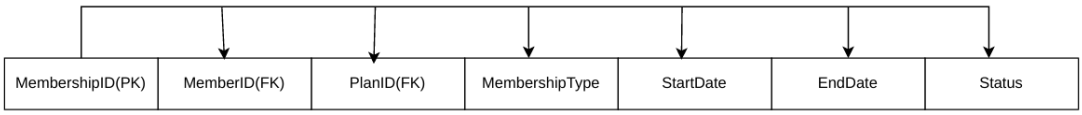



CLASS(CLASS_ID,CLASS_NAME,MAX_CAPACITY)

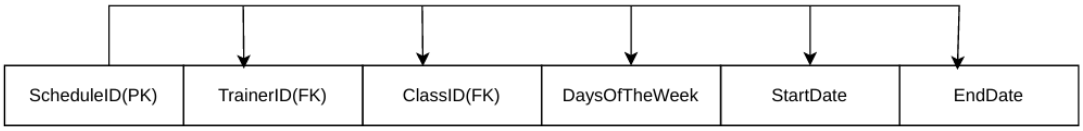



*Transitive Dependency:* *MemberID → StarRanking*

GYM_ATTENDANCE(ATTENDANE_ID,MEMBER_ID,STAR_RANKING,ENTRY_TIME,ATTENDANCE_DATE)




MEMBERSHIP(MEMBERSHIP_ID,MEMBER_ID,PLAN_ID,MEMBERSHIP_TYPE,START_DATE,END_DATE,STATUS)

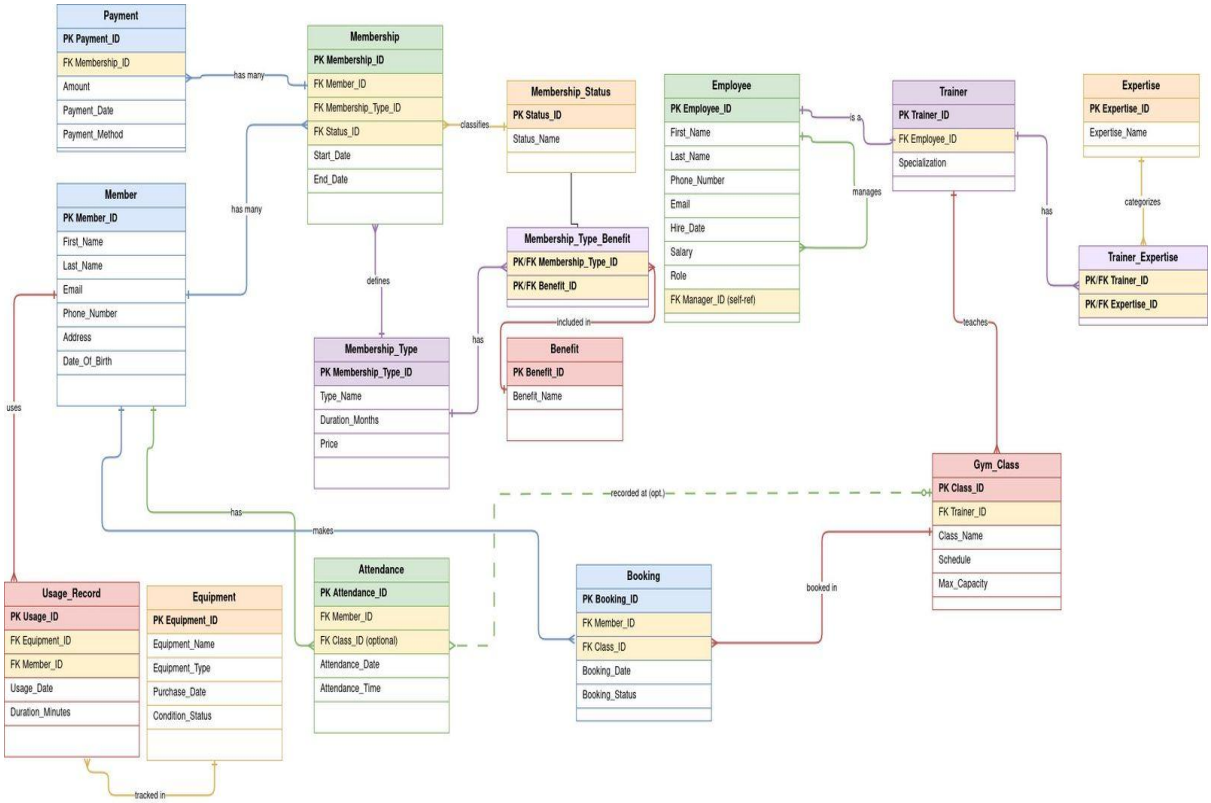



SCHEDULE(SCHEDULE_ID,TRAINER_ID,CLASS_ID,DAYS_OF_THE_WEEK,START_DATE,END_DATE)

# Project Phase 3 – Physical Design

## 1. Database objects

Creation of the initial database and further creation of the relationships within various tables within the database. Once the database has been formed, the database will then be populated with records which will then be used to demonstrate various queries throughout our database.




## Tables

/* DROP OLD OBJECTS */

DROP TABLE Trainer_Expertise CASCADE CONSTRAINTS;

DROP TABLE Membership_Type_Benefit CASCADE CONSTRAINTS;

DROP TABLE Usage_Record CASCADE CONSTRAINTS;

DROP TABLE Attendance CASCADE CONSTRAINTS;

DROP TABLE Booking CASCADE CONSTRAINTS;

DROP TABLE Payment CASCADE CONSTRAINTS;

DROP TABLE Gym_Class CASCADE CONSTRAINTS;

```
DROP TABLE Trainer CASCADE CONSTRAINTS;
DROP TABLE Expertise CASCADE CONSTRAINTS;
DROP TABLE Equipment CASCADE CONSTRAINTS;
DROP TABLE Membership CASCADE CONSTRAINTS;
DROP TABLE Membership_Type CASCADE CONSTRAINTS;
DROP TABLE Membership_Status CASCADE CONSTRAINTS;
DROP TABLE Benefit CASCADE CONSTRAINTS;
DROP TABLE Employee CASCADE CONSTRAINTS;
DROP TABLE MEMBER CASCADE CONSTRAINTS;

DROP SEQUENCE membership_seq;
DROP SEQUENCE booking_seq;
DROP SEQUENCE payment_seq;
DROP SEQUENCE attendance_seq;
DROP SEQUENCE usage_record_seq;



/* MEMBER TABLE */

CREATE TABLE MEMBER (
    Member_ID NUMBER(5) PRIMARY KEY,
    First_Name VARCHAR2(50) NOT NULL,
    Last_Name VARCHAR2(50) NOT NULL,
    Email VARCHAR2(100) UNIQUE,
    Phone_Number VARCHAR2(15) UNIQUE,
    Address VARCHAR2(150),
    Date_Of_Birth DATE,

    /* Extra attributes from business rules */
```

```
    Gender CHAR(1),
    Star_Ranking NUMBER(1) DEFAULT 0,

    CONSTRAINT chk_gender
        CHECK (Gender IN ('M', 'F')),

    CONSTRAINT chk_star_ranking
        CHECK (Star_Ranking BETWEEN 0 AND 5)
);


/* MEMBERSHIP_STATUS TABLE */

CREATE TABLE Membership_Status (
    Status_ID NUMBER(5) PRIMARY KEY,
    Status_Name VARCHAR2(30) NOT NULL UNIQUE
);


/* MEMBERSHIP_TYPE TABLE */

CREATE TABLE Membership_Type (
    Membership_Type_ID NUMBER(5) PRIMARY KEY,
    Type_Name VARCHAR2(50) NOT NULL UNIQUE,
    Duration_Months NUMBER(3) NOT NULL,
    Price NUMBER(8,2) NOT NULL,

    CONSTRAINT chk_duration_months
        CHECK (Duration_Months > 0),
```

```
    CONSTRAINT chk_price
        CHECK (Price > 0)
);


/* MEMBERSHIP TABLE */

CREATE TABLE Membership (
    Membership_ID NUMBER(5) PRIMARY KEY,
    Member_ID NUMBER(5) NOT NULL,
    Membership_Type_ID NUMBER(5) NOT NULL,
    Status_ID NUMBER(5) NOT NULL,
    Start_Date DATE NOT NULL,
    End_Date DATE NOT NULL,

    CONSTRAINT chk_membership_dates
        CHECK (End_Date >= Start_Date),

    CONSTRAINT fk_membership_member
        FOREIGN KEY (Member_ID)
        REFERENCES MEMBER(Member_ID),

    CONSTRAINT fk_membership_type
        FOREIGN KEY (Membership_Type_ID)
        REFERENCES Membership_Type(Membership_Type_ID),

    CONSTRAINT fk_membership_status
        FOREIGN KEY (Status_ID)
```

```
        REFERENCES Membership_Status(Status_ID)
);


/* BENEFIT TABLE */

CREATE TABLE Benefit (
    Benefit_ID NUMBER(5) PRIMARY KEY,
    Benefit_Name VARCHAR2(100) NOT NULL UNIQUE
);


/* MEMBERSHIP_TYPE_BENEFIT TABLE */

CREATE TABLE Membership_Type_Benefit (
    Membership_Type_ID NUMBER(5),
    Benefit_ID NUMBER(5),

    CONSTRAINT pk_membership_type_benefit
        PRIMARY KEY (Membership_Type_ID, Benefit_ID),

    CONSTRAINT fk_mtb_membership_type
        FOREIGN KEY (Membership_Type_ID)
        REFERENCES Membership_Type(Membership_Type_ID),

    CONSTRAINT fk_mtb_benefit
        FOREIGN KEY (Benefit_ID)
        REFERENCES Benefit(Benefit_ID)
);
```

```
/* EMPLOYEE TABLE */

CREATE TABLE Employee (
    Employee_ID NUMBER(5) PRIMARY KEY,
    First_Name VARCHAR2(50) NOT NULL,
    Last_Name VARCHAR2(50) NOT NULL,
    Phone_Number VARCHAR2(15) UNIQUE,
    Email VARCHAR2(100) UNIQUE,
    Hire_Date DATE NOT NULL,
    Salary NUMBER(10,2),
    Role VARCHAR2(50),
    Manager_ID NUMBER(5),

    CONSTRAINT chk_salary
        CHECK (Salary > 0),

    CONSTRAINT chk_employee_role
        CHECK (Role IN
        ('Trainer', 'Manager',
         'Receptionist', 'Cleaner')),

    CONSTRAINT fk_employee_manager
        FOREIGN KEY (Manager_ID)
        REFERENCES Employee(Employee_ID)
);
```

```
/* TRAINER TABLE */

CREATE TABLE Trainer (
    Trainer_ID NUMBER(5) PRIMARY KEY,
    Employee_ID NUMBER(5) UNIQUE NOT NULL,
    Specialization VARCHAR2(100),

    CONSTRAINT fk_trainer_employee
        FOREIGN KEY (Employee_ID)
        REFERENCES Employee(Employee_ID)
);


/* EXPERTISE TABLE */

CREATE TABLE Expertise (
    Expertise_ID NUMBER(5) PRIMARY KEY,
    Expertise_Name VARCHAR2(100) NOT NULL UNIQUE
);


/* TRAINER_EXPERTISE TABLE */

CREATE TABLE Trainer_Expertise (
    Trainer_ID NUMBER(5),
    Expertise_ID NUMBER(5),

    CONSTRAINT pk_trainer_expertise
        PRIMARY KEY (Trainer_ID, Expertise_ID),
```

```
    CONSTRAINT fk_te_trainer
        FOREIGN KEY (Trainer_ID)
        REFERENCES Trainer(Trainer_ID),

    CONSTRAINT fk_te_expertise
        FOREIGN KEY (Expertise_ID)
        REFERENCES Expertise(Expertise_ID)
);



/* GYM_CLASS TABLE */

CREATE TABLE Gym_Class (
    Class_ID NUMBER(5) PRIMARY KEY,
    Trainer_ID NUMBER(5) NOT NULL,
    Class_Name VARCHAR2(100) NOT NULL UNIQUE,
    Schedule VARCHAR2(100),
    Max_Capacity NUMBER(3) NOT NULL,

    CONSTRAINT chk_max_capacity
        CHECK (Max_Capacity > 0),

    CONSTRAINT fk_class_trainer
        FOREIGN KEY (Trainer_ID)
        REFERENCES Trainer(Trainer_ID)
);
```

```
/* EQUIPMENT TABLE */

CREATE TABLE Equipment (
    Equipment_ID NUMBER(5) PRIMARY KEY,
    Equipment_Name VARCHAR2(100) NOT NULL,
    Equipment_Type VARCHAR2(50),
    Purchase_Date DATE,
    Condition_Status VARCHAR2(30) DEFAULT 'GOOD',

    CONSTRAINT chk_condition_status
        CHECK (Condition_Status IN
        ('GOOD', 'DAMAGED', 'UNDER MAINTENANCE'))
);


/* USAGE_RECORD TABLE */

CREATE TABLE Usage_Record (
    Usage_ID NUMBER(5) PRIMARY KEY,
    Equipment_ID NUMBER(5) NOT NULL,
    Member_ID NUMBER(5) NOT NULL,
    Usage_Date DATE DEFAULT SYSDATE,
    Duration_Minutes NUMBER(5),

    CONSTRAINT chk_duration_minutes
        CHECK (Duration_Minutes > 0),

    CONSTRAINT fk_usage_equipment
        FOREIGN KEY (Equipment_ID)
```

```
        REFERENCES Equipment(Equipment_ID),

    CONSTRAINT fk_usage_member
        FOREIGN KEY (Member_ID)
        REFERENCES MEMBER(Member_ID)
);


/* ATTENDANCE TABLE */

CREATE TABLE Attendance (
    Attendance_ID NUMBER(5) PRIMARY KEY,
    Member_ID NUMBER(5) NOT NULL,
    Class_ID NUMBER(5),
    Attendance_Date DATE NOT NULL,
    Attendance_Time TIMESTAMP NOT NULL,

    CONSTRAINT fk_attendance_member
        FOREIGN KEY (Member_ID)
        REFERENCES MEMBER(Member_ID),

    CONSTRAINT fk_attendance_class
        FOREIGN KEY (Class_ID)
        REFERENCES Gym_Class(Class_ID)
);


/* BOOKING TABLE */
```

```
CREATE TABLE Booking (
    Booking_ID NUMBER(5) PRIMARY KEY,
    Member_ID NUMBER(5) NOT NULL,
    Class_ID NUMBER(5) NOT NULL,
    Booking_Date DATE NOT NULL,
    Booking_Status VARCHAR2(30) DEFAULT 'PENDING',

    CONSTRAINT chk_booking_status
        CHECK (Booking_Status IN
        ('CONFIRMED', 'PENDING', 'CANCELLED')),

    CONSTRAINT fk_booking_member
        FOREIGN KEY (Member_ID)
        REFERENCES MEMBER(Member_ID),

    CONSTRAINT fk_booking_class
        FOREIGN KEY (Class_ID)
        REFERENCES Gym_Class(Class_ID)
);


/* PAYMENT TABLE */

CREATE TABLE Payment (
    Payment_ID NUMBER(5) PRIMARY KEY,
    Member_ID NUMBER(5) NOT NULL,
    Amount NUMBER(10,2) NOT NULL,
    Payment_Date DATE NOT NULL,
    Payment_Method VARCHAR2(50) NOT NULL,
```

```
    CONSTRAINT chk_payment_amount
        CHECK (Amount >= 0),

    CONSTRAINT chk_payment_method
        CHECK (Payment_Method IN
        ('Cash', 'Card', 'EFT')),

    CONSTRAINT fk_payment_member
        FOREIGN KEY (Member_ID)
        REFERENCES MEMBER(Member_ID)
);


/* SEQUENCES TABLE */

CREATE SEQUENCE membership_seq
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;

CREATE SEQUENCE booking_seq
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;

CREATE SEQUENCE payment_seq
```

```
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;

CREATE SEQUENCE attendance_seq
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;

CREATE SEQUENCE usage_record_seq
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;


/* MEMBERSHIP TRIGGER TABLE */

CREATE OR REPLACE TRIGGER membership_trigger
BEFORE INSERT ON Membership
FOR EACH ROW
BEGIN
   IF :NEW.Membership_ID IS NULL THEN
      SELECT membership_seq.NEXTVAL
      INTO :NEW.Membership_ID
      FROM DUAL;
   END IF;
```

```
END;
/

/* BOOKING TRIGGER TABLE */

CREATE OR REPLACE TRIGGER booking_trigger
BEFORE INSERT ON Booking
FOR EACH ROW
BEGIN
   IF :NEW.Booking_ID IS NULL THEN
      SELECT booking_seq.NEXTVAL
      INTO :NEW.Booking_ID
      FROM DUAL;
   END IF;
END;
/

/* PAYMENT TRIGGER TABLE */

CREATE OR REPLACE TRIGGER payment_trigger
BEFORE INSERT ON Payment
FOR EACH ROW
BEGIN
   IF :NEW.Payment_ID IS NULL THEN
      SELECT payment_seq.NEXTVAL
      INTO :NEW.Payment_ID
      FROM DUAL;
   END IF;
END;
```

```
/

/* ATTENDANCE TRIGGER TABLE */

CREATE OR REPLACE TRIGGER attendance_trigger
BEFORE INSERT ON Attendance
FOR EACH ROW
BEGIN
   IF :NEW.Attendance_ID IS NULL THEN
      SELECT attendance_seq.NEXTVAL
      INTO :NEW.Attendance_ID
      FROM DUAL;
   END IF;
END;
/

/* USAGE_RECORD TRIGGER TABLE */

CREATE OR REPLACE TRIGGER usage_record_trigger
BEFORE INSERT ON Usage_Record
FOR EACH ROW
BEGIN
   IF :NEW.Usage_ID IS NULL THEN
      SELECT usage_record_seq.NEXTVAL
      INTO :NEW.Usage_ID
      FROM DUAL;
   END IF;
END;
/
```

## Populate Tables

```
SET DEFINE OFF;


/* MEMBER */

INSERT INTO MEMBER
(Member_ID, First_Name, Last_Name, Email,
 Phone_Number, Address, Date_Of_Birth,
 Gender, Star_Ranking)
VALUES
(1, 'John', 'Smith', 'john.smith@email.com',
'0712340001', '12 Oak Street, Johannesburg',
TO_DATE('1990-05-14','YYYY-MM-DD'),
'M', 4);

INSERT INTO MEMBER
(Member_ID, First_Name, Last_Name, Email,
 Phone_Number, Address, Date_Of_Birth,
 Gender, Star_Ranking)
VALUES
(2, 'Sarah', 'Johnson', 'sarah.johnson@email.com',
'0712340002', '45 Pine Avenue, Pretoria',
TO_DATE('1988-11-22','YYYY-MM-DD'),
'F', 5);

INSERT INTO MEMBER
(Member_ID, First_Name, Last_Name, Email,
 Phone_Number, Address, Date_Of_Birth,
```

```
 Gender, Star_Ranking)
VALUES
(3, 'Michael', 'Brown', 'michael.brown@email.com',
'0712340003', '78 Cedar Road, Durban',
TO_DATE('1992-03-09','YYYY-MM-DD'),
'M', 3);

INSERT INTO MEMBER
(Member_ID, First_Name, Last_Name, Email,
 Phone_Number, Address, Date_Of_Birth,
 Gender, Star_Ranking)
VALUES
(4, 'Emily', 'Davis', 'emily.davis@email.com',
'0712340004', '23 Maple Drive, Cape Town',
TO_DATE('1995-07-18','YYYY-MM-DD'),
'F', 4);

INSERT INTO MEMBER
(Member_ID, First_Name, Last_Name, Email,
 Phone_Number, Address, Date_Of_Birth,
 Gender, Star_Ranking)
VALUES
(5, 'Daniel', 'Wilson', 'daniel.wilson@email.com',
'0712340005', '90 Birch Lane, Bloemfontein',
TO_DATE('1987-01-30','YYYY-MM-DD'),
'M', 5);


/* MEMBERSHIP_STATUS */
```

```
INSERT INTO Membership_Status VALUES (1, 'Active');
INSERT INTO Membership_Status VALUES (2, 'Expired');
INSERT INTO Membership_Status VALUES (3, 'Cancelled');
INSERT INTO Membership_Status VALUES (4, 'Suspended');
INSERT INTO Membership_Status VALUES (5, 'Pending');


/* MEMBERSHIP_TYPE */

INSERT INTO Membership_Type VALUES
(1, 'Basic', 12, 299.99);

INSERT INTO Membership_Type VALUES
(2, 'Premium', 12, 499.99);

INSERT INTO Membership_Type VALUES
(3, 'VIP', 12, 799.99);

INSERT INTO Membership_Type VALUES
(4, 'Student', 6, 149.99);

INSERT INTO Membership_Type VALUES
(5, 'Day Pass', 1, 19.99);


/* BENEFIT */

INSERT INTO Benefit VALUES (1, 'Gym Access');
```

```
INSERT INTO Benefit VALUES (2, 'Locker Room Access');
INSERT INTO Benefit VALUES (3, 'Group Classes');
INSERT INTO Benefit VALUES (4, 'Sauna Access');
INSERT INTO Benefit VALUES (5, 'Personal Trainer Sessions');


/* EMPLOYEE */

INSERT INTO Employee
(Employee_ID, First_Name, Last_Name,
 Phone_Number, Email, Hire_Date,
 Salary, Role, Manager_ID)
VALUES
(1, 'Michael', 'Adams',
'0715551001', 'michael.adams@gym.com',
TO_DATE('2023-01-15','YYYY-MM-DD'),
35000, 'Trainer', NULL);

INSERT INTO Employee
(Employee_ID, First_Name, Last_Name,
 Phone_Number, Email, Hire_Date,
 Salary, Role, Manager_ID)
VALUES
(2, 'Sarah', 'Williams',
'0715551002', 'sarah.williams@gym.com',
TO_DATE('2023-03-10','YYYY-MM-DD'),
34000, 'Trainer', NULL);

INSERT INTO Employee
```

```
(Employee_ID, First_Name, Last_Name,
 Phone_Number, Email, Hire_Date,
 Salary, Role, Manager_ID)
VALUES
(3, 'David', 'Nkosi',
'0715551003', 'david.nkosi@gym.com',
TO_DATE('2023-04-05','YYYY-MM-DD'),
36000, 'Trainer', NULL);

INSERT INTO Employee
(Employee_ID, First_Name, Last_Name,
 Phone_Number, Email, Hire_Date,
 Salary, Role, Manager_ID)
VALUES
(4, 'Jessica', 'Brown',
'0715551004', 'jessica.brown@gym.com',
TO_DATE('2023-06-20','YYYY-MM-DD'),
33000, 'Trainer', NULL);

INSERT INTO Employee
(Employee_ID, First_Name, Last_Name,
 Phone_Number, Email, Hire_Date,
 Salary, Role, Manager_ID)
VALUES
(5, 'Chris', 'Daniels',
'0715551005', 'chris.daniels@gym.com',
TO_DATE('2023-08-15','YYYY-MM-DD'),
34500, 'Trainer', NULL);
```

```
/* TRAINER */

INSERT INTO Trainer
(Trainer_ID, Employee_ID, Specialization)
VALUES
(1, 1, 'Strength Training');

INSERT INTO Trainer
(Trainer_ID, Employee_ID, Specialization)
VALUES
(2, 2, 'Weight Loss & Cardio');

INSERT INTO Trainer
(Trainer_ID, Employee_ID, Specialization)
VALUES
(3, 3, 'Bodybuilding');

INSERT INTO Trainer
(Trainer_ID, Employee_ID, Specialization)
VALUES
(4, 4, 'Yoga & Flexibility');

INSERT INTO Trainer
(Trainer_ID, Employee_ID, Specialization)
VALUES
(5, 5, 'CrossFit & HIIT');
```

```
/* EXPERTISE */

INSERT INTO Expertise VALUES (1, 'Strength Training');
INSERT INTO Expertise VALUES (2, 'Weight Loss');
INSERT INTO Expertise VALUES (3, 'Cardio');
INSERT INTO Expertise VALUES (4, 'Bodybuilding');
INSERT INTO Expertise VALUES (5, 'Yoga');


/* EQUIPMENT */

INSERT INTO Equipment VALUES
(1, 'Treadmill', 'Cardio',
TO_DATE('2024-01-10','YYYY-MM-DD'),
'GOOD');

INSERT INTO Equipment VALUES
(2, 'Bench Press', 'Strength',
TO_DATE('2024-02-15','YYYY-MM-DD'),
'GOOD');

INSERT INTO Equipment VALUES
(3, 'Exercise Bike', 'Cardio',
TO_DATE('2024-03-01','YYYY-MM-DD'),
'GOOD');

INSERT INTO Equipment VALUES
(4, 'Elliptical Machine', 'Cardio',
TO_DATE('2024-01-20','YYYY-MM-DD'),
```

```
'GOOD');

INSERT INTO Equipment VALUES
(5, 'Squat Rack', 'Strength',
TO_DATE('2024-02-10','YYYY-MM-DD'),
'GOOD');


/* MEMBERSHIP */

INSERT INTO Membership
(Member_ID, Membership_Type_ID,
 Status_ID, Start_Date, End_Date)
VALUES
(1, 1, 1,
TO_DATE('2025-01-01','YYYY-MM-DD'),
TO_DATE('2025-12-31','YYYY-MM-DD'));

INSERT INTO Membership
(Member_ID, Membership_Type_ID,
 Status_ID, Start_Date, End_Date)
VALUES
(2, 2, 1,
TO_DATE('2025-01-02','YYYY-MM-DD'),
TO_DATE('2025-12-31','YYYY-MM-DD'));

INSERT INTO Membership
(Member_ID, Membership_Type_ID,
 Status_ID, Start_Date, End_Date)
```

```
VALUES
(3, 3, 1,
TO_DATE('2025-01-03','YYYY-MM-DD'),
TO_DATE('2025-12-31','YYYY-MM-DD'));

INSERT INTO Membership
(Member_ID, Membership_Type_ID,
 Status_ID, Start_Date, End_Date)
VALUES
(4, 1, 1,
TO_DATE('2025-01-04','YYYY-MM-DD'),
TO_DATE('2025-12-31','YYYY-MM-DD'));

INSERT INTO Membership
(Member_ID, Membership_Type_ID,
 Status_ID, Start_Date, End_Date)
VALUES
(5, 2, 1,
TO_DATE('2025-01-05','YYYY-MM-DD'),
TO_DATE('2025-12-31','YYYY-MM-DD'));


/* MEMBERSHIP_TYPE_BENEFIT */

INSERT INTO Membership_Type_Benefit VALUES (1,1);
INSERT INTO Membership_Type_Benefit VALUES (1,2);

INSERT INTO Membership_Type_Benefit VALUES (2,1);
INSERT INTO Membership_Type_Benefit VALUES (2,2);
```

```
INSERT INTO Membership_Type_Benefit VALUES (2,3);

INSERT INTO Membership_Type_Benefit VALUES (3,1);
INSERT INTO Membership_Type_Benefit VALUES (3,2);
INSERT INTO Membership_Type_Benefit VALUES (3,3);
INSERT INTO Membership_Type_Benefit VALUES (3,4);
INSERT INTO Membership_Type_Benefit VALUES (3,5);


/* TRAINER_EXPERTISE */

INSERT INTO Trainer_Expertise VALUES (1,1);
INSERT INTO Trainer_Expertise VALUES (2,2);
INSERT INTO Trainer_Expertise VALUES (2,3);
INSERT INTO Trainer_Expertise VALUES (3,4);
INSERT INTO Trainer_Expertise VALUES (4,5);


/* GYM_CLASS */

INSERT INTO Gym_Class
(Class_ID, Trainer_ID, Class_Name,
 Schedule, Max_Capacity)
VALUES
(1, 4, 'Yoga',
'Monday 08:00-09:00', 20);

INSERT INTO Gym_Class
(Class_ID, Trainer_ID, Class_Name,
```

```
 Schedule, Max_Capacity)
VALUES
(2, 4, 'Pilates',
'Monday 09:00-10:00', 15);

INSERT INTO Gym_Class
(Class_ID, Trainer_ID, Class_Name,
 Schedule, Max_Capacity)
VALUES
(3, 5, 'HIIT Training',
'Tuesday 10:00-11:00', 25);

INSERT INTO Gym_Class
(Class_ID, Trainer_ID, Class_Name,
 Schedule, Max_Capacity)
VALUES
(4, 1, 'Strength & Conditioning',
'Wednesday 12:00-13:00', 20);

INSERT INTO Gym_Class
(Class_ID, Trainer_ID, Class_Name,
 Schedule, Max_Capacity)
VALUES
(5, 2, 'Zumba',
'Wednesday 13:00-14:00', 35);


/* BOOKING */
```

```
INSERT INTO Booking
(Member_ID, Class_ID,
 Booking_Date, Booking_Status)
VALUES
(1, 1,
TO_DATE('2025-05-01','YYYY-MM-DD'),
'CONFIRMED');

INSERT INTO Booking
(Member_ID, Class_ID,
 Booking_Date, Booking_Status)
VALUES
(2, 2,
TO_DATE('2025-05-01','YYYY-MM-DD'),
'CONFIRMED');

INSERT INTO Booking
(Member_ID, Class_ID,
 Booking_Date, Booking_Status)
VALUES
(3, 3,
TO_DATE('2025-05-01','YYYY-MM-DD'),
'PENDING');

INSERT INTO Booking
(Member_ID, Class_ID,
 Booking_Date, Booking_Status)
VALUES
(4, 4,
```

```
TO_DATE('2025-05-02','YYYY-MM-DD'),
'CANCELLED');

INSERT INTO Booking
(Member_ID, Class_ID,
 Booking_Date, Booking_Status)
VALUES
(5, 5,
TO_DATE('2025-05-02','YYYY-MM-DD'),
'CONFIRMED');


/* PAYMENT */

INSERT INTO Payment
(Member_ID, Amount,
 Payment_Date, Payment_Method)
VALUES
(1, 299.99,
TO_DATE('2025-01-01','YYYY-MM-DD'),
'Card');

INSERT INTO Payment
(Member_ID, Amount,
 Payment_Date, Payment_Method)
VALUES
(2, 499.99,
TO_DATE('2025-01-02','YYYY-MM-DD'),
'EFT');
```

```
INSERT INTO Payment
(Member_ID, Amount,
 Payment_Date, Payment_Method)
VALUES
(3, 799.99,
TO_DATE('2025-01-03','YYYY-MM-DD'),
'Cash');

INSERT INTO Payment
(Member_ID, Amount,
 Payment_Date, Payment_Method)
VALUES
(4, 299.99,
TO_DATE('2025-01-04','YYYY-MM-DD'),
'Card');

INSERT INTO Payment
(Member_ID, Amount,
 Payment_Date, Payment_Method)
VALUES
(5, 499.99,
TO_DATE('2025-01-05','YYYY-MM-DD'),
'EFT');


/* ATTENDANCE */

INSERT INTO Attendance
```

```
(Member_ID, Class_ID,
 Attendance_Date, Attendance_Time)
VALUES
(1, 1,
TO_DATE('2025-05-01','YYYY-MM-DD'),
TO_TIMESTAMP('2025-05-01 06:00:00',
'YYYY-MM-DD HH24:MI:SS'));

INSERT INTO Attendance
(Member_ID, Class_ID,
 Attendance_Date, Attendance_Time)
VALUES
(2, 2,
TO_DATE('2025-05-01','YYYY-MM-DD'),
TO_TIMESTAMP('2025-05-01 06:15:00',
'YYYY-MM-DD HH24:MI:SS'));

INSERT INTO Attendance
(Member_ID, Class_ID,
 Attendance_Date, Attendance_Time)
VALUES
(3, 3,
TO_DATE('2025-05-01','YYYY-MM-DD'),
TO_TIMESTAMP('2025-05-01 06:30:00',
'YYYY-MM-DD HH24:MI:SS'));

INSERT INTO Attendance
(Member_ID, Class_ID,
 Attendance_Date, Attendance_Time)
```

```
VALUES
(4, NULL,
TO_DATE('2025-05-02','YYYY-MM-DD'),
TO_TIMESTAMP('2025-05-02 07:00:00',
'YYYY-MM-DD HH24:MI:SS'));

INSERT INTO Attendance
(Member_ID, Class_ID,
 Attendance_Date, Attendance_Time)
VALUES
(5, 5,
TO_DATE('2025-05-02','YYYY-MM-DD'),
TO_TIMESTAMP('2025-05-02 07:30:00',
'YYYY-MM-DD HH24:MI:SS'));


/* USAGE_RECORD */

INSERT INTO Usage_Record
(Equipment_ID, Member_ID,
 Usage_Date, Duration_Minutes)
VALUES
(1, 1,
TO_DATE('2025-05-01','YYYY-MM-DD'),
45);

INSERT INTO Usage_Record
(Equipment_ID, Member_ID,
 Usage_Date, Duration_Minutes)
```

```
VALUES
(2, 2,
TO_DATE('2025-05-01','YYYY-MM-DD'),
30);

INSERT INTO Usage_Record
(Equipment_ID, Member_ID,
 Usage_Date, Duration_Minutes)
VALUES
(3, 3,
TO_DATE('2025-05-02','YYYY-MM-DD'),
60);

INSERT INTO Usage_Record
(Equipment_ID, Member_ID,
 Usage_Date, Duration_Minutes)
VALUES
(4, 4,
TO_DATE('2025-05-02','YYYY-MM-DD'),
40);

INSERT INTO Usage_Record
(Equipment_ID, Member_ID,
 Usage_Date, Duration_Minutes)
VALUES
(5, 5,
TO_DATE('2025-05-03','YYYY-MM-DD'),
50);
```

```
COMMIT;
```

## Queries

```
/** 1. Member Profile **/
SELECT First_Name, Last_Name, Phone_Number FROM MEMBER;

/** 2. Trainer Expertise **/
SELECT Trainer_ID, Specialization FROM Trainer;

/** 3. Active Membership Check **/
SELECT m.First_Name, m.Last_Name, ms.End_Date
FROM MEMBER m
JOIN Membership ms ON m.Member_ID = ms.Member_ID
WHERE ms.End_Date > SYSDATE;

/** 4. Emergency Contact List **/
SELECT First_Name, Last_Name, Address, Phone_Number FROM MEMBER;

/** 5. Role Breakdown **/
SELECT Role, COUNT(*) AS Employee_Count FROM Employee GROUP BY Role;

/** 6. Class Capacity Status **/
SELECT Class_Name, Max_Capacity FROM Gym_Class;

/** 7. Payment Audit **/
SELECT Payment_ID, Payment_Method, Amount, Payment_Date FROM Payment;

/** 8. Trainer Load **/
SELECT t.Trainer_ID, gc.Class_Name
FROM Trainer t
JOIN Gym_Class gc ON t.Trainer_ID = gc.Trainer_ID;
```

```
/** 9. Benefit Catalog for Premium **/
SELECT b.Benefit_Name
FROM Benefit b
JOIN Membership_Type_Benefit mtb ON b.Benefit_ID = mtb.Benefit_ID
WHERE mtb.Membership_Type_ID = 2;

/** 10. Equipment Condition **/
SELECT Equipment_Name, Condition_Status FROM Equipment;

/** 1. Recent Joiners **/
SELECT First_Name, Email FROM MEMBER WHERE ROWNUM <= 10 ORDER BY Member_ID DESC;

/** 2. Top Paid Staff **/
SELECT First_Name, Last_Name, Salary FROM Employee WHERE ROWNUM <= 3 ORDER BY Salary DESC;

/** 3. Quick Booking View **/
SELECT Member_ID, Booking_Date FROM Booking WHERE ROWNUM <= 20;

/** 4. Limited Equipment View **/
SELECT Equipment_Name FROM Equipment WHERE ROWNUM <= 5;

/** 1. Schedule Timeline **/
SELECT Class_Name, Schedule FROM Gym_Class ORDER BY Schedule;

/** 2. Member Birthdays **/
SELECT First_Name, Last_Name, Date_Of_Birth FROM MEMBER ORDER BY Date_Of_Birth;
```

/** 3. Highest Payments **/

SELECT Payment_ID, Amount FROM Payment ORDER BY Amount DESC;

/** 4. Experience Sort **/

SELECT First_Name, Last_Name, Hire_Date FROM Employee ORDER BY Hire_Date;

/** 1. Strength Focus **/

SELECT Class_Name FROM Gym_Class WHERE Class_Name LIKE '%Strength%';

/** 2. Specific Providers **/

SELECT First_Name, Last_Name, Email FROM MEMBER WHERE Email LIKE '%gmail.com' OR Email LIKE '%outlook.com';

/** 3. High-Value/Long-Term **/

SELECT Type_Name, Price, Duration_Months FROM Membership_Type WHERE Price > 500 AND Duration_Months = 12;

/** 4. Search by Initial **/

SELECT First_Name, Last_Name FROM Employee WHERE First_Name LIKE 'S%';

/** 1. Formal Name **/

SELECT CONCAT(CONCAT(Last_Name, ', '), First_Name) AS Formal_Name FROM MEMBER;

/** 2. Email Cleanup **/

SELECT LOWER(Email) AS Clean_Email FROM MEMBER;

/** 3. Short Specialization **/

```
SELECT SUBSTR(Specialization, 1, 10) AS Short_Specialization FROM Trainer;

/** 4. Username Generation **/

SELECT SUBSTR(First_Name, 1, 3) || Member_ID AS Username FROM MEMBER;

/** 1. Daily Equivalent **/

SELECT Price, ROUND(Price / 30, 2) AS Daily_Equivalent FROM
Membership_Type;

/** 2. Staff Salary Truncation **/

SELECT First_Name, Last_Name, TRUNC(Salary) AS Truncated_Salary FROM
Employee;

/** 3. Average Class Duration **/

SELECT AVG(Duration_Minutes) AS Avg_Minutes,
ROUND(AVG(Duration_Minutes)/60, 1) AS Avg_Hours FROM Usage_Record;

/** 4. VAT Calculation **/

SELECT Amount, ROUND(Amount * 0.15, 2) AS VAT, ROUND(Amount * 1.15, 2) AS
Total_Inc_VAT FROM Payment;

/** 1. Membership Aging **/

SELECT Membership_ID, (End_Date - Start_Date) AS Days_Active FROM
Membership;

/** 2. Payment Month **/

SELECT Payment_ID, TO_CHAR(Payment_Date, 'Month') AS Payment_Month
FROM Payment;

/** 3. Contract Reminders **/

SELECT m.First_Name, m.Last_Name, ms.End_Date
```

```
FROM MEMBER m
JOIN Membership ms ON m.Member_ID = ms.Member_ID
WHERE ms.End_Date = SYSDATE + 7;

/** 4. Attendance Day **/
SELECT TO_CHAR(Attendance_Date, 'DY') AS Day_Name, COUNT(*) AS
Attendance_Count
FROM Attendance
GROUP BY TO_CHAR(Attendance_Date, 'DY')
ORDER BY Attendance_Count DESC;

/** 1. Total Revenue **/
SELECT SUM(Amount) AS Total_Revenue FROM Payment;

/** 2. Active Member Count **/
SELECT COUNT(DISTINCT m.Member_ID) AS Active_Members
FROM MEMBER m
JOIN Membership ms ON m.Member_ID = ms.Member_ID
WHERE ms.End_Date > SYSDATE;

/** 3. Average Member Age **/
SELECT AVG(MONTHS_BETWEEN(SYSDATE, Date_Of_Birth)/12) AS Avg_Age
FROM MEMBER;

/**  4. Minimum Price **/
SELECT MIN(Price) AS Cheapest_Plan_Price FROM Membership_Type;

/** 1. Popularity Check **/
SELECT gc.Class_Name, COUNT(b.Booking_ID) AS Booking_Count
FROM Gym_Class gc
```

```
LEFT JOIN Booking b ON gc.Class_ID = b.Class_ID
GROUP BY gc.Class_Name
ORDER BY Booking_Count DESC;

/** 2. Staffing Needs (Peak Hours) **/
SELECT TO_CHAR(Attendance_Time, 'HH24') AS Hour, COUNT(*) AS
Attendance_Count
FROM Attendance
GROUP BY TO_CHAR(Attendance_Time, 'HH24')
ORDER BY Attendance_Count DESC;

/** 3. High-Activity Members **/
SELECT Member_ID, COUNT(*) AS Visit_Count
FROM Attendance
GROUP BY Member_ID
HAVING COUNT(*) > 15;

/** 4. Revenue by Method **/
SELECT Payment_Method, SUM(Amount) AS Total_Amount
FROM Payment
GROUP BY Payment_Method
ORDER BY Total_Amount DESC;

/** 1. Booking Confirmation (Today's Bookings) **/
SELECT m.First_Name, m.Last_Name, gc.Class_Name, b.Booking_Date
FROM Booking b
JOIN MEMBER m ON b.Member_ID = m.Member_ID
JOIN Gym_Class gc ON b.Class_ID = gc.Class_ID
WHERE TRUNC(b.Booking_Date) = TRUNC(SYSDATE);
```

```
/** 2. Trainer Class List (Yoga) **/
SELECT t.Trainer_ID, e.First_Name, e.Last_Name, gc.Class_Name
FROM Trainer t
JOIN Employee e ON t.Employee_ID = e.Employee_ID
JOIN Gym_Class gc ON t.Trainer_ID = gc.Trainer_ID
WHERE gc.Class_Name = 'Yoga';

/** 3. Member Plan Details **/
SELECT m.First_Name, m.Last_Name, mt.Type_Name, mt.Price
FROM MEMBER m
JOIN Membership ms ON m.Member_ID = ms.Member_ID
JOIN Membership_Type mt ON ms.Membership_Type_ID =
mt.Membership_Type_ID;

/** 4. Usage Audit **/
SELECT e.Equipment_Name, m.First_Name, m.Last_Name, ur.Duration_Minutes
FROM Usage_Record ur
JOIN Equipment e ON ur.Equipment_ID = e.Equipment_ID
JOIN MEMBER m ON ur.Member_ID = m.Member_ID
ORDER BY ur.Duration_Minutes DESC;

/** 5. Benefit Access for Member ID 1 **/
SELECT DISTINCT b.Benefit_Name
FROM MEMBER m
JOIN Membership ms ON m.Member_ID = ms.Member_ID
JOIN Membership_Type mt ON ms.Membership_Type_ID =
mt.Membership_Type_ID
JOIN Membership_Type_Benefit mtb ON mt.Membership_Type_ID =
mtb.Membership_Type_ID
```

```
JOIN Benefit b ON mtb.Benefit_ID = b.Benefit_ID

WHERE m.Member_ID = 1;


/** 1. Underpaid Trainers **/

SELECT e.First_Name, e.Last_Name, e.Salary

FROM Employee e

JOIN Trainer t ON e.Employee_ID = t.Employee_ID

WHERE e.Salary < (SELECT AVG(Salary) FROM Employee);


/** 2. Ghost Members (Paid but Zero Attendance) **/

SELECT DISTINCT m.Member_ID, m.First_Name, m.Last_Name

FROM MEMBER m

JOIN Payment p ON m.Member_ID = p.Member_ID

WHERE m.Member_ID NOT IN (SELECT DISTINCT Member_ID FROM
Attendance);


/** 3. Max Attendance Streak (Star Ranking Candidate) **/

SELECT Member_ID, COUNT(*) AS Attendance_Count

FROM Attendance

GROUP BY Member_ID

HAVING COUNT(*) = (SELECT MAX(COUNT(*)) FROM Attendance GROUP BY
Member_ID);


/** 4. Over-capacity Risk **/

SELECT gc.Class_ID, gc.Class_Name, gc.Max_Capacity, COUNT(b.Booking_ID)
AS Bookings

FROM Gym_Class gc

JOIN Booking b ON gc.Class_ID = b.Class_ID

GROUP BY gc.Class_ID, gc.Class_Name, gc.Max_Capacity

HAVING COUNT(b.Booking_ID) = gc.Max_Capacity;
```

```
/** 5. Recent Active Payers (Paid and Attended Same Month) **/
SELECT DISTINCT m.Member_ID, m.First_Name, m.Last_Name
FROM MEMBER m
WHERE EXISTS (
   SELECT 1 FROM Payment p
   WHERE p.Member_ID = m.Member_ID
   AND EXISTS (
      SELECT 1 FROM Attendance a
      WHERE a.Member_ID = m.Member_ID
      AND TO_CHAR(a.Attendance_Date, 'YYYY-MM') =
TO_CHAR(p.Payment_Date, 'YYYY-MM')
   )
);
```

## Add Ons

```
/** INDEXES **/

CREATE INDEX idx_membership_member ON Membership(Member_ID);
CREATE INDEX idx_membership_type ON Membership(Membership_Type_ID);
CREATE INDEX idx_membership_status ON Membership(Status_ID);
CREATE INDEX idx_booking_member ON Booking(Member_ID);
CREATE INDEX idx_booking_class ON Booking(Class_ID);
CREATE INDEX idx_booking_date ON Booking(Booking_Date);
CREATE INDEX idx_attendance_member ON Attendance(Member_ID);
CREATE INDEX idx_attendance_class ON Attendance(Class_ID);
CREATE INDEX idx_attendance_date ON Attendance(Attendance_Date);
CREATE INDEX idx_payment_member ON Payment(Member_ID);
CREATE INDEX idx_payment_date ON Payment(Payment_Date);
```

```
CREATE INDEX idx_usage_member ON Usage_Record(Member_ID);
CREATE INDEX idx_usage_equipment ON Usage_Record(Equipment_ID);
CREATE INDEX idx_gym_class_trainer ON Gym_Class(Trainer_ID);
CREATE INDEX idx_member_email ON MEMBER(Email);
CREATE INDEX idx_member_phone ON MEMBER(Phone_Number);
CREATE INDEX idx_membership_end_date ON Membership(End_Date);

/** VIEWS **/

CREATE OR REPLACE VIEW Active_Members_View AS
SELECT m.Member_ID, m.First_Name, m.Last_Name, m.Email, m.Phone_Number,
    mt.Type_Name AS Membership_Type, ms.Start_Date, ms.End_Date
FROM MEMBER m
JOIN Membership ms ON m.Member_ID = ms.Member_ID
JOIN Membership_Type mt ON ms.Membership_Type_ID =
mt.Membership_Type_ID
WHERE ms.End_Date > SYSDATE AND ms.Status_ID = 1;

CREATE OR REPLACE VIEW Class_Schedule_View AS
SELECT gc.Class_ID, gc.Class_Name, gc.Schedule, gc.Max_Capacity,
    e.First_Name AS Trainer_First, e.Last_Name AS Trainer_Last,
    t.Specialization
FROM Gym_Class gc
JOIN Trainer t ON gc.Trainer_ID = t.Trainer_ID
JOIN Employee e ON t.Employee_ID = e.Employee_ID;

CREATE OR REPLACE VIEW Member_Payment_Summary AS
SELECT m.Member_ID, m.First_Name, m.Last_Name,
    COUNT(p.Payment_ID) AS Total_Payments,
```

```
    SUM(p.Amount) AS Total_Spent,
    MAX(p.Payment_Date) AS Last_Payment_Date
FROM MEMBER m
LEFT JOIN Payment p ON m.Member_ID = p.Member_ID
GROUP BY m.Member_ID, m.First_Name, m.Last_Name;

/** EXTRA FUNCTIONALITY: Capacity Trigger **/

CREATE OR REPLACE TRIGGER check_booking_capacity
BEFORE INSERT ON Booking
FOR EACH ROW
DECLARE
  v_current_count NUMBER;
  v_max_capacity NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_current_count
  FROM Booking
  WHERE Class_ID = :NEW.Class_ID
  AND Booking_Status = 'CONFIRMED';

  SELECT Max_Capacity INTO v_max_capacity
  FROM Gym_Class
  WHERE Class_ID = :NEW.Class_ID;

  IF v_current_count >= v_max_capacity THEN
    RAISE_APPLICATION_ERROR(-20001, 'Class is full! Cannot book.');
  END IF;
END;
/
```

```
/** EXTRA FUNCTIONALITY: Renewal Procedure **/

CREATE OR REPLACE PROCEDURE Renew_Membership(
    p_Member_ID IN NUMBER,
    p_New_Membership_Type_ID IN NUMBER
) AS
    v_duration_months NUMBER;
BEGIN
    /** Get duration of new membership **/
    SELECT Duration_Months INTO v_duration_months
    FROM Membership_Type
    WHERE Membership_Type_ID = p_New_Membership_Type_ID;

    /** Update existing membership to Expired **/
    UPDATE Membership
    SET Status_ID = 2
    WHERE Member_ID = p_Member_ID AND Status_ID = 1;

    /** Insert new membership **/
    INSERT INTO Membership (Member_ID, Membership_Type_ID, Status_ID,
Start_Date, End_Date)
    VALUES (p_Member_ID, p_New_Membership_Type_ID, 1, SYSDATE,
ADD_MONTHS(SYSDATE, v_duration_months));

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Membership renewed for Member ID: ' ||
p_Member_ID);
END;
/
```